

HelixQA

HelixQA

反虚假测试质量保证编排——跨平台自主会话，每次通过均附带真实用户可操作功能的实证记录。

概要

HelixQA 是一套反虚假测试质量保证编排框架，适用于跨平台测试（Android、Android TV、Web、桌面端），结合 YAML 测试库、实时崩溃检测、逐步实证记录捕获，以及基于 LLM 与计算机视觉的自主质量保证会话，以验证功能真正实现端到端运行。该框架为 Constitution 规定的强制性质量保证测试类型 (§11.4.169)。

简要说明

反虚假测试质量保证编排器 (Go) 可执行预设测试库，并运行跨平台的全自主、基于 LLM 与视觉驱动的质量保证会话——检测崩溃、逐步验证（截图、logcat、视频、堆栈跟踪等实证记录），并自动生成附带丰富证据的工单，供 AI 修复流水线使用。

详细说明

HelixQA 是一套 Go 框架，其核心设计理念坚守 Constitution §11.4 运行规则：发布标准并非“测试通过”，而是“用户能够使用该功能”，因此其每次“通过”结果均需附带执行期间捕获的实时证据——无证据即无通过，绝无例外。该框架运行两种互补模式，全面覆盖脚本化与未知场景。

首先，预设测试库——由 YAML 提供的 TC-XXX 测试套件，包含平台定向、优先级、有序步骤（名称/操作/预期结果）、标签及文档引用，执行时逐步验证，实时检测崩溃/ANR（Android 通过 ADB，Web/桌面端通过进程监控），集中收集证据，并自动生成符合下游 AI 修复流水线格式的 Markdown 工单。

其次，全自主质量保证会话——将应用交由 LLM 驱动的智能体与计算机视觉接管，无人值守运行，分为四个严谨阶段：准备（选择大语言模型、基于项目文档构建功能地图、启动 CLI 智能体、初始化视觉引擎）、文档驱动验证（遍历所有已文档化功能）、好奇心驱动探索（主动测试边界情况及未文档化行为），最后生成包含视频时间戳证据链接的 Markdown/HTML/JSON 报告并完成清理。

关键在于，该框架不自行评判结果：它集成了四个外部 Go 子模块（LLMsVerifier、LLMOrchestrator、VisionEngine、DocProcessor），并复用共享的 challenges 与 containers 基础设施，确保应用导航组件与结果判定组件相互独立。其自身测试套件同样遵循与被测产品相同的标准，通过 make anti-bluff（静态扫描 + 行为锚点清单 + 变异棘轮）及内置 §1.1 变异的 8 阶段编排挑战进行验证。一份包含 15 行测试类型覆盖矩阵的文档，将每项宣称的功能与具体可执行资产及特定证据形式严格对应——框架对自身的承诺，与其对被测产品的判定一样，均需以实证为依据。

内容

我们为何构建它

传统的质量保证（QA）仅以「断言通过」为准绿灯放行，而Constitution所称的虚假通过——即功能在报告中显示正常，实则对真实用户失效——正是因此得以蒙混过关。HelixQA的诞生，正是为了从根本上杜绝这一现象：它拒绝在缺乏实际执行证据（截图、logcat日志、视频、堆栈跟踪、报告）的情况下给出「通过」评分，并将无证据支撑的绿色汇总行视为与缺失功能同等严重的缺陷。同时，它还解决了人力瓶颈——全面的人工QA无法跨平台规模化——通过实现完全自主的测试流程。

为何它是变革性的

它将两种几乎从未在同一工具中并存的能力融为一体：严谨的证据驱动QA门控，与自主、自驱动的探索。一个集成LLM视觉模块的智能体能够打开真实应用，验证每一项文档记载的功能，主动搜寻未被任何测试用例覆盖的隐藏缺陷，并在过程中生成可用于法庭质证级别的证据链——「我们测试过」从此被「这是视频、这是日志、这是工单」所取代。而由于它是Constitution命名的QA子模块，采用它并非仅为单个团队提升QA诚信度——而是通过一次部署，为整个产品家族的所有衍生产品设定更高的质量基准。

创新之处

- 反虚假通过证据契约：每项检查的「通过」必须绑定捕获的运行时证据；CI中的绿色状态线虽为必要条件，但绝非充分条件，无证据支撑的绿色汇总将被判定为严重缺陷。
- 自主文档驱动 + 好奇心驱动探索：它不仅验证所有文档记载的功能，还会脱离脚本，探测真实用户可能遭遇的边界情况（空输入、快速交互、未文档化的路径）——这些都是手写测试套件无法预见的。

- 视觉预言机：GoCV机器视觉与LLM视觉模块API能够实时「看见」屏幕上的UI，捕捉令基于令牌或属性的断言束手无策的视觉异常。
- 结构化而非文本化的测试库：测试库中的字符串描述结构而非文本，在运行时驱动LLM生成问题提示（CONST-046），因此单一测试库可跨语言环境使用，无需因UI文本翻译而支离破碎。
- 为AI修复流水线定制的工单：自动生成的Markdown工单附带完整证据包，可直接交付下游修复智能体处理，无需人工分拣。

如何在所有产品中应用（它赋予的能力）

作为强制质量支柱（Constitution §11.4.169将helix_qa子模块列为必备测试类型之一），HelixQA为家族中的每个产品赋予了相同的能力：

- 自主QA会话：单条命令
helixqa autonomous --project ... --platforms android,desktop,web
即可释放一个集成LLM视觉模块的智能体，在无人值守的情况下驱动真实应用完成覆盖目标，并输出报告、工单和视频。
- 测试库/套件：YAML测试库（第219轮迭代，基准 ≥ 30 ），针对特定平台优先级排序，且每行均可追溯至其验证的文档。
- 捕获证据：截图、logcat日志、视频、堆栈跟踪及完整时间线——集中存储并链接至每份报告，确保任何判定均可事后重现与审核。
- 独立判定（§11.4.141独立性原则）：其基于LLM的issuедetector与视觉预言机独立评判运行中应用的行为，结构性排除了「系统自证清白」的经典失效模式。
- 门控 + 变异棘轮：make qa-all / make anti-bluff及challenges/scripts/helixqa_orchestrator_challenge.sh（8阶段，内置§1.1变异机制）持续验证HelixQA自身的诚信度——且刻意未提供--skip-helixqa逃生通道，以确保在截止日期压力下亦无法关闭这一质量约束。

内容

最大的技术挑战及解决方案

- 避免 QA 工具本身出现误报——检测虚假信息工具绝不能成为虚假信息的源头 → 每一步都需通过捕获的证据验证，缺乏证据的「通过」将被判定为缺陷而非合格，且行为锚定清单（CONST-035）将每项宣称的功能与可执行测试绑定，确保任何功能宣称都有实际测试支撑。
- 用单一大脑驱动异构平台——Android、Android TV、Web 和桌面平台的输入模型各不相同 → 统一的 navigator 包抽象了平台特定的 ActionExecutors（ADB、Playwright、X11）及各平台崩溃检测器（Android/Web/桌面），使编排逻辑只需编写一次，平台差异则被隔离在边界处。
- 让自主代理有用而非混乱——未受监督的 LLM 在应用中可能无休止地漫游 → LLMsVerifier 负责评分和选择合适的模型，LLMOrchestrator 管理无界面的 CLI 代理（opencode、claude-code、gemini、junie、qwen-code），DocProcessor 构建特征地图为探索提供目标，VisionEngine 则确保每个决策基于屏幕上的真实像素而非模型的臆想。

- 支持本地化的测试库——硬编码英文 UI 文本的测试套件在十五种语言环境下会失效 → 测试库仅描述结构，用户可见的提示文本在运行时通过 LLM/资源加载 (CONST-046)，确保同一测试库能验证相同行为，无论语言环境如何变化。
- 证明防虚假检查机制并非摆设——无法自证失败的防虚假检查机制本身就是最大的虚假 → 成对的 §1.1 变异测试会剥离类型的证据捕获或防虚假断言，要求检查机制必须失败，且变异棘轮机制防止这一保障随时间悄然弱化。

技术栈

- **Go 1.24+ 编排器**
 - 原因：QA 工具必须能在产品运行的任何环境中运行，因此单一静态链接、快速且可移植的二进制文件优于依赖运行时的替代方案；
 - 实现：一个 cmd/helixqa CLI，提供可组合的子命令 `run / list / report / autonomous / version`。
- **YAML 测试库 (pkg/testbank)**
 - 原因：测试套件应具备声明式和可读性，无需修改 Go 即可由人工编辑；
 - 实现：包含 `version/name/test_cases[]`，每个测试用例拥有 `id`、`category`、`priority`、`platforms`、有序的 `steps[]` 及 `documentation_refs[]`，以确保可追溯至功能文档。
- **崩溃/ANR 检测器 (pkg/detector)**
 - 原因：最关键的失败往往发生在交互过程中，而非事后断言；
 - 实现：Android 使用 ADB (`pidof/logcat/screenshot`)，Web/桌面使用 `pgrep`，在测试驱动过程中实时监控进程。
- **证据采集 (pkg/evidence、pkg/session)**
 - 原因：防虚假合约只有在每个「通过」结果都有实物证据支撑时才真实有效；
 - 实现：通过 `SessionRecorder` 时间线捕获截图、`logcat`、视频及堆栈跟踪，每份报告均可回溯至证据。
- **自主会话 (pkg/autonomous、pkg/navigator、pkg/issuedetector)**
 - 原因：在四个平台上进行全面的人工 QA 难以扩展，探索过程本身必须能够自动驱动；
 - 实现：四阶段的 `SessionCoordinator`，结合 `ActionExecutors` (ADB/Playwright/X11) 及 LLM 缺陷检测，覆盖视觉、UX、无障碍及功能性缺陷。
- **外部子模块**
 - 原因：复用与解耦 (CONST-051)，且——关键在于——将导航器与判定器分离；
 - 实现：LLMsVerifier (模型评分)、LLMOrchestrator (无界面 CLI 代理)、VisionEngine (GoCV + LLM Vision)、DocProcessor (特征地图/覆盖率)，每个组件独立维护。
- **防虚假检查机制 + 变异棘轮**
 - 原因：确保 HelixQA 严格遵守其对其他组件强制执行的 §1.1 约定；
 - 实现：make anti-bluff 扫描，结合行为锚定清单和变异棘轮机制，并通过 `helixqa_orchestrator_challenge.sh` 作为 8 阶段端到端验证器。

- **15 行覆盖率矩阵 (docs/test-coverage.md)**
 - 原因：CONST-050(B) 要求覆盖类型集必须封闭且无遗漏；
 - 实现：每行绑定具体可执行资产及特定的证据捕获形式，确保覆盖率是可验证的事实而非主观宣称。

内容

状态与诚信说明

- 状态：测试版。处于积极开发阶段（README 状态横幅第 219 轮）。严守自身反虚假承诺标准。
- 许可证：**Apache-2.0**。安装命令：`go install digital.vasic.helixqa/cmd/helixqa@latest`。

优先级别：Helix-主线 —— Helix 系列验证功能是否真正可用的强制性质量/反虚假承诺支柱。